



LECTURE – 16

Phase – 4: Software Testing

Testing, Blackbox Testing

Suleman Shahid

Also Based on

Dr. Maryam Abdul Ghafoor

CS 360 - Software Engineering (Spring 2024)

LAHORE UNIVERSITY OF
MANAGEMENT SCIENCES



1

Software Testing — Warm-Up Quiz

Before we begin: no Googling, go with your gut!

2

“50 % of my employees are testers, and the rest
spends 50% of the time on testing”
– Bill Gates

3

IEEE.ORG IEEE XPLORE DIGITAL LIBRARY IEEE STANDARDS MORE SITES

IEEE Spectrum FOR THE TECHNOLOGY INSIDER

FEATURE AEROSPACE

HOW THE BOEING 737 MAX DISASTER LOOKS TO A SOFTWARE DEVELOPER

Design shortcuts meant to make a new plane seem like an old, familiar one are to blame

4



5

Boeing 737 MAX — 346 Lives Lost (2018–2019)

- Two Boeing 737 MAX aircraft crashed, Lion Air Flight 610 and Ethiopian Airlines Flight 302, killing 346 people. **The global fleet was grounded for 20 months.**
- **The bug:** A flight control system called MCAS (Maneuvering Characteristics Augmentation System) was designed to prevent stalling. It relied on a single angle-of-attack sensor. When that sensor gave faulty readings, MCAS repeatedly pushed the nose of the aircraft down. Pilots were not adequately informed the system existed, let alone trained to override it.
- **The lesson:** A system that makes critical decisions based on a single unvalidated sensor, with no redundancy and no pilot override education, is a system waiting to fail. **Failure mode testing, what happens when input data is wrong, is not optional.**

6

Software Fault, Error and Failures

- **Software Fault:** A static defect in the software
- **Software Failure:** External, incorrect behavior with respect to the requirements or other description of the expected behavior
- **Software Error:** An incorrect internal state that is the manifestation of some fault

Faults in software are equivalent to design mistakes in hardware.

Software Defects

- Human errors can lead to faults in work products, which may cause failures when running the software
- Can try to find faults through testing, reviews, proof, model checking, code analysis, etc.
- Some avoid the term bug, since it implies something wandered into the code

Examples of Defects

- Actual behavior differing from expected:
 - Algorithmic
 - Code logic does not produce the proper output
 - Overload
 - Data structure unexpectedly filled completely
 - Performance
 - Violates service level agreement
 - Accuracy
 - Calculated result not to the desired level of accuracy
 - Timing
 - Race condition in coordinating concurrent processes

Failure

- AT&T failure (1990):
 - 114 switching nodes of their long-distance system crashed •
 - The outage lasted for 9 h, 70 million calls went uncompleted
- Reason:
 - If a node crashes, it tells neighboring nodes to reroute traffic around it
 - A bug in handling this message caused the receiving node to also crash, etc

Software Fault, Failure and Error

Function: returns double of the number

```
1. int doubleValue(int i)
2. {
3.     int double;
4.     double = i * i;
5.     return double;
6. }
```

CS360 Spring 2026

LUMS

11

Better?

```
int doubleValue(int i)
{
    int result;
    result = i * 2;
    return result;
}
```

CS360 Spring 2026

LUMS

12

Better?

```
int doubleValue(int i)
{
    return i * 2;
}
```

Example

Function: If array is null throw NullPointerException, else return the number of occurrences of 0 in array

```
public static int numZero (int[ ] array){
    int count = 0;
    for (int i = 1; i < array.length; i++){
        if (array[ i ] == 0){
            count++;
        }
    }
    return count;
}
```

Arrays are ----- indexed, so any loop intending to traverse the whole array should start at ----?

```
public static int numZero(int[] array) {
    int count = 0;
    for (int i = 0; i < array.length; i++) {
        if (array[i] == 0) {
            count++;
        }
    }
    return count;
}
```

CS360 Spring 2026

LUMS

15

Fault in Code? Root Cause...

```
do {
    switch (...) {
        case ...:
            if (...) {
                ...
                break;
            } else {
                ...
            }
            ...
    }
} while (...);
```

CS360 Spring 2026

LUMS

16


```

• do {
•   switch {
•     case:
•       if {
•         ...
•         break; ← exits switch only, NOT the do-while
•       } else {
•         ...
•       }
•     ...
•   }
• } while (...);

```

← if break fired, this is skipped but do-while still continues!

CS360 Spring 2026

LUMS

17

Fault in Code? Root Cause...

```

do {
  switch (...) {
    case ...:
      if (...) {
        ...
        break;
      } else {
        ...
      }
      ...
  }
} while (...);

```

The key issue here is **what does break break out of?**
 A common misconception is that **break inside the if** exits the do-while loop. It does **not**, it only breaks out of the **switch**, and the do-while continues.

CS360 Spring 2026

LUMS

18

If the intent was to break the loop, the fix would be to use a flag:

- `bool done = false;`
- `do {`
- `switch (...) {`
- `case ...:`
- `if (...) {`
- `...`
- `done = true; // signal loop to stop`
- `} else {`
- `...`
- `}`
- `...`
- `}`
- `} while (!done);`

This is a classic **scoping trap** — `break` always exits the **nearest enclosing switch or loop**, which may not be what the programmer intended.

Why testing?

- Goals:
 - Verification
 - Check that requirements are satisfied
 - Not only to confirm normal behavior
 - Find problems to refute that the program is correct
 - Establish due diligence
 - Evidence in case of product liability litigation
 - Avoid regression
 - Prevent previous problems from reoccurring

Limits of Testing

- Issues:
 - A program cannot be tested completely
 - Too many inputs and path combinations to cover
 - Testing cannot find all defects
 - Cannot show their absence, just their presence
 - Challenging
 - Testing may be expensive and frustrating
 - Test code itself could add its own defects

CS360 Spring 2026

LUMS

21

Test Driven Development

- More on this in the next class
- Including Automated testing

CS360 Spring 2026

LUMS

22

Types of Testing

Type	What it tests	Who does it
Unit Testing	Individual functions/methods in isolation	Developers
Integration Testing	Interactions between modules/components	Developers / QA
System Testing	Entire system against the specification	QA team
Acceptance Testing	Whether system meets user/business needs	Client / end users
Regression Testing	That fixes don't break existing behaviour	QA / CI pipeline
Performance Testing	Speed, scalability, stability under load	Specialists
Security Testing	Vulnerabilities and attack surfaces	Security teams

Testing Approaches

- Testing
- Verification
- Inspections
- Formal proofs of correctness

Testing and Verification



Test case = $\{i \in I, o \in O\}$

Test suite = set of test cases

No false positives

incomplete

Verification



Considers all possible inputs/executions

Can have false positives

Inspections and Formal Verification

- **Inspections**

- Reviews of the code
- Walkthroughs
- Code is inspected manually
- Group activity
- Systematic, thorough
- Informal, subjective



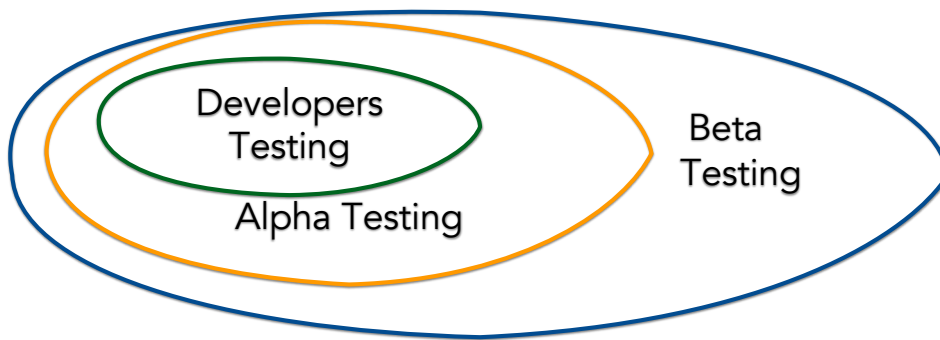
- **Formal Verification**

- Formal proof of correctness proofs specification
- Sound guarantees,
- Complex, expensive

A Test is successful if the program fails

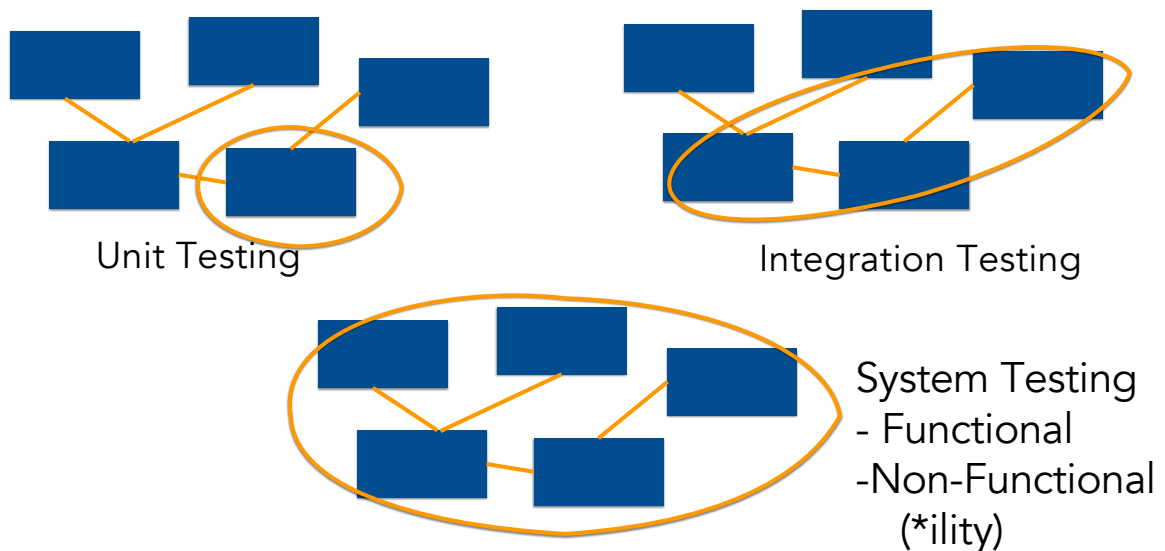
Towards a Theory of Test data Selection (IEEE TSE-85)
Goodenough and Gerhart

Software Testing

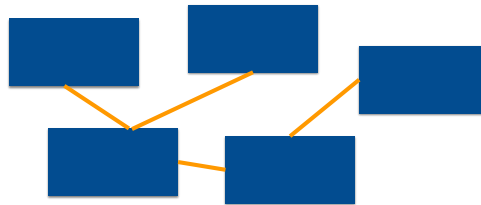


- Alpha Testing is done by the people in organization
- Beta Testing is with external users of the system

Software Testing (Developers' Testing)

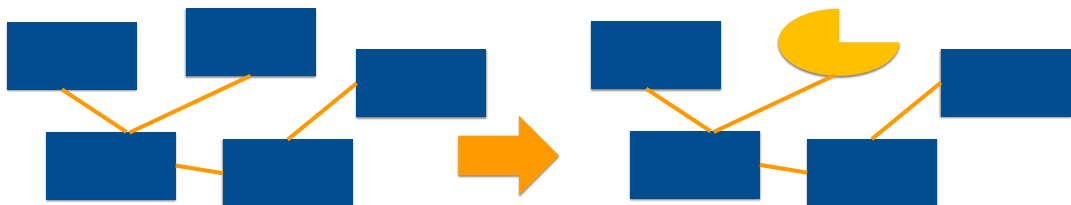


Software Testing



Acceptance Testing

Regression Testing

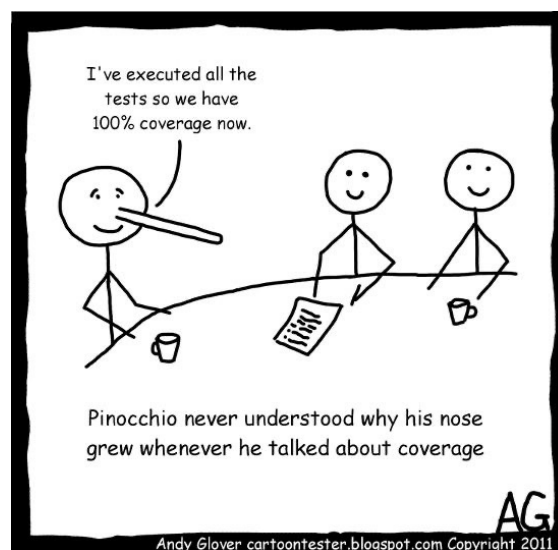


CS360 Spring 2026

LUMS

31

Test Coverage



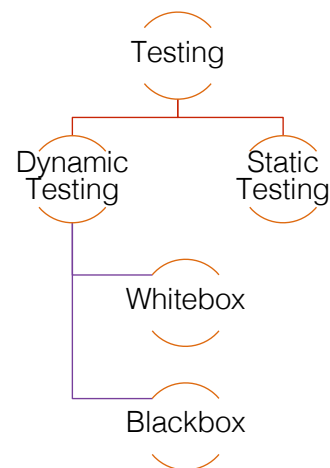
CS360 Spring 2026

LUMS

32

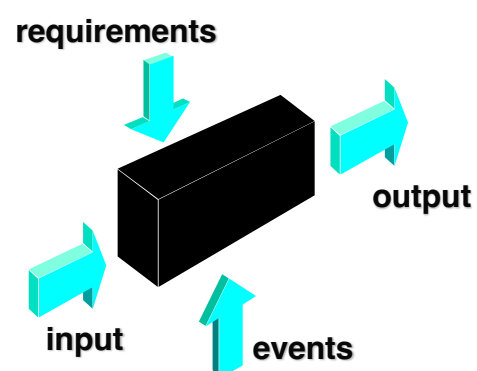
Software Testing

- Executing a program on a sample input to uncover undesired behavior
 - Dynamic Technique
 - Optimistic approximation



Black-Box Testing – Functional testing

- Based on the description of the software specification (SRS)
- Covers specified behavior
- Can not reveal errors due to implementation details



Whitebox Testing

- Based on the code
- Covers as much coded behavior as possible
- Can not reveal errors due to missing paths

Blackbox vs Whitebox Testing

Specification: a function that inputs an integer and prints it

Implementation:

```
void printNum(int num) {  
    if (num < 1024) cout << num;  
    else cout << num/104 << "KB";  
}
```

Blackbox vs Whitebox Testing

Implementation:

```
void printNum(int num) {
    cout << num*num*num;
}
```

Specification: Function prints the cube of the function, if it is even number, otherwise square of it

Implication

- If your test suite **only uses even numbers**, the bug goes completely undetected. This illustrates why **test case selection matters**, you must test both even and odd inputs to adequately cover the specification.
- This is precisely why coverage criteria (boundary cases, equivalence partitioning) exist to ensure **you don't accidentally only test the happy path**.

The interesting testing implication here:

This bug can produce **false positives** — tests that pass even though the code is wrong:

Input	Buggy output	Correct output	Test result
2 (even)	8	8	✓ passes — but for the wrong reason
4 (even)	64	64	✓ passes — but for the wrong reason
3 (odd)	27	9	✗ caught

Correct?

```
void printNum(int num)
{
    if (num % 2 == 0)
        cout << num * num * num; // even → cube
    else
        cout << num * num; // odd → square
}
```

CS360 Spring 2026

LUMS

40

Writing Test Cases

LAHORE UNIVERSITY OF
MANAGEMENT SCIENCES

41

Test Oracle

- Test Oracle is the set of test cases representing desired behavior of the program on the given input

SL.No	Test Case	Expected Result	Test Result
1	On the click of ADD button	At first user have to fill all fields with proper data , if any Error like entering text data instead of number or entering number instead of text..is found then it gives proper message otherwise Adds Record To the Database	successful
2.	On the Click of DELETE Button	This deletes the details of student by using Register no.	Successful
3.	On the Click of UPDATE Button	Modified records are Updated in database by clicking UPDATE button.	Successful
4.	On the Click of SEARCH Button	Displays the Details of book for entered Register no. Otherwise gives proper Error message.	Successful
5.	On the Click of CLEAR	Clears all fields	Successful

CS360 Spring 2026

LUMS

42

Test Log

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TU01	Check Customer Login with valid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	Userid = guru99 Password = pass99	User should Login into an application	As Expected	Pass
TU02	Check Customer Login with invalid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	Userid = guru99 Password = glass99	User should not Login into an application	As Expected	Pass

CS360 Spring 2026

LUMS

43

Thank you!